

REPORT ABOUT TRANSACTION CONTROL AND ISOLATION LEVELS

Prepared by:

B2205896

Nguyễn Minh Nhựt

B2205976

Trần Hữu Giang

B2203438

Huỳnh Tấn Đạt

B2205983

Đỗ Đạt Hoa

Instructor: Trần Công Ân

Course: Oracle Database Management System

1. Transaction.....	3
1.1. What is Transaction in DBMS?.....	3
2. Transaction Control.....	3
2.1. Define.....	3
2.2. Commands are used to control the transaction.....	3
3. Transaction Isolation.....	4
3.1. Oracle transaction isolation level.....	4
3.1.1. READ COMMITTED.....	4
3.1.2. SERIALIZABLE.....	5
3.1.3. READ ONLY.....	5
4. Example.....	5
4.1. READ COMMITTED.....	5
4.2. SERIALIZABLE.....	6
4.3. READ ONLY.....	6
4.4. Summary.....	7
5. The advantages and disadvantages of each isolation level...	7
5.1. READ COMMITTED.....	7
5.2. SERIALIZABLE.....	8
5.3. READ ONLY.....	8
6. References.....	9

1.Transaction

1.1. What is Transaction in DBMS?

A transaction is a unit of program execution that accesses and can update various data items. Typically, a transaction is initiated by a user program written in a data manipulation language (often SQL) or a programming language with embedded database access rights using JDBC or ODBC. A transaction includes all operations executed between the start and the end of the transaction.

To ensure data integrity and consistency, transactions must conform to ACID properties:

- **Atomicity:** Either all the operations of a transaction are executed correctly in the database, or none of them are executed.
- **Consistency:** Executing a transaction in isolation (i.e., without any other transactions running simultaneously) will ensure the consistency of the database.
- **Isolation:** Each transaction is unaffected by other concurrently running transactions as if they are executed sequentially.
- **Durability:** Once a transaction is completed, its changes are permanently saved, regardless of any system failure.

2. Transaction Control

2.1. Define

Transaction control is the process of managing transactions to ensure data integrity and consistency. It ensures that changes in the database are only applied when a transaction is successfully completed, and it allows for rollback in case a transaction fails or encounters an error.

2.2. Commands are used to control the transaction.

COMMIT: This command confirms all changes made by a transaction and saves them to the database. After a COMMIT, the changes become permanent and cannot be rolled back.

- **Syntax:** COMMIT;

SAVEPOINT: SAVEPOINT is a point in the transaction where transaction can be rolled back without entire transaction rollback.

- **Syntax:** SAVEPOINT savepoint_name;

ROLLBACK: The ROLLBACK command is used to undo saved changes made by the transaction in the database.

- **Syntax:** ROLLBACK;
-- Or roll back to a savepoint
ROLLBACK TO SAVEPOINT savepoint_name;

SET TRANSACTION: Sets the properties of a transaction, such as isolation level and read-only mode.

- **Syntax:** SET TRANSACTION [ISOLATION LEVEL level];

3. Transaction Isolation

Transaction isolation provides the application with a view as if the system processes one transaction at a time, even though there are multiple simultaneous connections to the database. Applications can use the general connection property **Isolation** to set the isolation level for a connection.

Transaction Isolation Levels are mechanisms that ensure that concurrently running transactions do not cause conflicts or lose database consistency. Depending on the isolation level, the balance between consistency and performance can be adjusted to suit the specific application.

Three phenomena that violate serializability:

- **Dirty read:** The transaction reads values written by another transaction that hasn't been committed yet.
- **Non-repeatable read:** A transaction reads the same object twice during execution and finds a different value the second time, although the transaction has not changed the value in the meantime.
- **Phantom read:** A transaction re-executes a query returning a set of rows that satisfy a search condition and finds that the set of rows satisfying the condition has changed as a result of another recently committed transaction.

3.1. Oracle transaction isolation level

Oracle, like many other databases, uses the concept of isolation levels to describe how a transaction will interact with other transactions and how it will be isolated from other transactions. An isolation level is essentially a locking scheme implemented by the database to ensure a certain type of transaction isolation.

The two basic isolation levels commonly used in Oracle are: **READ COMMITTED** (default isolation level of Oracle) and **SERIALIZABLE** (with a third level being **READ ONLY**). Both of these isolation levels produce database operations that can be serialized. The difference between the two levels lies in the timing of their serialization of operations

3.1.1. READ COMMITTED

- **Serialization is executed at the statement level.** This means that each statement will have a consistent view of the data as it existed at the start of that statement, ensuring that each transaction can only see the changes from other transactions after they have been COMMITted. However, since a transaction can contain multiple statements, phenomena such as **non-repeatable reads** and **phantom reads** can occur within the context of the entire transaction. The READ COMMITTED isolation level is the default isolation level in Oracle.

3.1.2. SERIALIZABLE

- **Serialization is executed at the transaction level.** This means that each statement within a transaction will have the same consistent view of the data as it existed at the start of the transaction.
- **SERIALIZABLE** is the highest isolation level in database transaction management, ensuring that transactions are executed as if they occurred sequentially, one after another (i.e., no transactions run concurrently). This provides the highest data integrity and prevents issues such as dirty reads, non-repeatable reads, and phantom reads, but it can impact system performance.

3.1.3. READ ONLY

- This isolation level prohibits write operations, ensuring that the session or transaction can only read data. Data will be retrieved as it existed at the start of the transaction, ensuring absolute consistency in reading the data without the risk of it being changed by other transactions.

4. Example

Here are specific examples of how transaction isolation levels work in Oracle with simple scenarios:

4.1. READ COMMITTED

Scenario:

- Assume there are two transactions accessing an **Accounts** table. This table has two columns: **Account_ID** and **Balance** (assume this account has **5000** dollars).
- Transaction A reads data from the **Accounts** table and then updates the balance but hasn't committed yet, while Transaction B makes changes (UPDATE) to the data and commits before Transaction A performs the query again

Example:

- **Transaction A** starts and executes the following query:

```
SELECT Balance FROM Accounts WHERE Account_ID = 1001;  
UPDATE Accounts SET Balance = Balance + 2000 WHERE  
Account_ID = 1001;
```

 - o **Balance: 5000** (The balance does not change because Transaction A hasn't been committed yet).
- **Transaction B** starts and updates the account balance:

```
UPDATE Accounts SET Balance = Balance + 3000 WHERE  
Account_ID = 1001;  
COMMIT;
```
- **Transaction A** continues to execute the same query again:

```
SELECT Balance FROM Accounts WHERE Account_ID = 1001;
```

 - o **Balance: 8000** (the balance has changed after Transaction B committed).

4.2. SERIALIZABLE

Scenario:

- The same two transactions A and B on the **Accounts** table, but this time Transaction A uses the **Serializable** isolation level.

Example:

- **Transaction A** starts with the **SERIALIZABLE** isolation level and executes a query:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
SELECT Balance FROM Accounts WHERE Account_ID = 1001;
```

 - o **Balance: 5000.**
- **Transaction B** starts and attempts to update the balance:

```
UPDATE Accounts SET Balance = Balance + 2000 WHERE  
Account_ID = 1001;  
COMMIT;
```

 - o If **Transaction A** has not yet finished and committed, **Transaction B** will be blocked or must wait until **Transaction A** completes. This ensures that **Transaction A** can be completed without being affected by changes from **Transaction B**.
- When **Transaction A** commits:

```
COMMIT;
```

 - o At this point, **Transaction B** can proceed with its commit.

4.3. READ ONLY

Scenario:

- The same two transactions A and B on the **Accounts** table, but this time Transaction A uses the **SERIALIZABLE** isolation level.

Example:

- **Transaction A** starts with the **READ ONLY** isolation level:

```
SET TRANSACTION READ ONLY;  
SELECT Balance FROM Accounts WHERE Account_ID = 1001;
```

 - Balance: **5000**.
 - Meanwhile, **Transaction B** changes the data and commits:

```
UPDATE Accounts SET Balance = Balance + 2000 WHERE  
Account_ID = 1001;  
COMMIT;
```
 - **Transaction A** continues to execute another query within the same transaction:

```
SELECT Balance FROM Accounts WHERE Account_ID = 1001;
```

 - **Balance: 5000** (the data is fixed at the time **READ ONLY** transaction starts, regardless of changes from other transactions).
-

4.4. Summary

- **READ COMMITTED:** Transactions can see changes committed by other transactions, with the possibility of encountering Non-repeatable Read and Phantom Read.
- **SERIALIZABLE:** Transactions are fully isolated, avoiding Dirty Read, Non-repeatable Read, and Phantom Read.
- **READ ONLY:** Transactions only read data that is consistent at the time the transaction starts and are not affected by any changes from other transactions.

5. The advantages and disadvantages of each isolation level.

5.1. READ COMMITTED

Advantages:

- **Better Performance:** Compared to stricter isolation levels (like **SERIALIZABLE**), **READ COMMITTED** offers better performance as it requires fewer data locks for shorter periods.
- **Reading Committed Data:** Ensures that a transaction only reads data that has been committed by other transactions, reducing the risk of reading uncommitted (inconsistent) data.
- **Higher Concurrency:** Works well in systems with high transaction concurrency, as it balances performance and data consistency.

Disadvantages:

- **Non-repeatable Reads:** A transaction might read a value, and if another transaction modifies that value and commits, a subsequent read in the same transaction may return a different result.
 - **Phantom Reads:** Another transaction can add or remove rows, which may lead to inconsistencies in read operations when executing queries more than once.
-

5.2. SERIALIZABLE

Advantages:

- **Highest Data Consistency:** **SERIALIZABLE** guarantees the highest level of isolation, ensuring that transactions are executed as if they were run sequentially, providing strong data consistency.
- **Prevents Non-repeatable and Phantom Reads:** No other transaction can modify or add data until the current transaction is completed, ensuring consistent results.

Disadvantages:

- **Lower Performance:** Because **SERIALIZABLE** requires locking entire tables or large sets of rows, it significantly reduces system concurrency and slows down performance.
 - **Resource Contention:** With more transactions waiting for locks to be released, it can cause delays and reduce throughput, especially in high-concurrency environments.
-

5.3. READ ONLY

Advantages:

- High Performance for Read Operations: Since READ ONLY transactions do not need to lock resources for writing, they are fast and efficient.
- Avoids Deadlocks and Contention: With no write operations, there are fewer chances for resource contention or deadlocks between transactions.
- Consistency: This guarantees that the transaction reads committed data and that the data won't change during the transaction.

Disadvantages:

- Cannot Perform Write Operations: READ ONLY transactions are limited to reading data and cannot perform updates, inserts, or deletes, limiting their applicability to certain tasks.
- Limited Use Case: Primarily suitable for reporting and analytics where only data reads are required, but not suitable for systems that need frequent updates or changes to the data.

6. References

Database System Concepts 6th edition

https://www.geeksforgeeks.org/transaction-control-in-dbms/?ref=oin_asr7

https://www.geeksforgeeks.org/transaction-management/?ref=oin_asr16

<https://blog.vietnamlab.vn/transaction-isolation-levels-trong-dbms/>

<https://learning.oreilly.com/library/view/oracle-essentials-oracle9i/0596001797/ch07s03.html>